AWS Cloud

Quick Prep Notes



Modules 1 - 9 | Java / Spring backend interview edition

Read this the night before the interview. Every topic: Definition > Why > How > Example > Trap.

Ink code: *Def* *Why* *How* *Eg* *TRAP* *key term*

Contents

Module 1	Cloud Foundations	p. 2
Module 2	IAM	p. 5
Module 3	EC2	p. 8
Module 4	S3	p. 11
Module 5	RDS and Databases	p. 14
Module 6	Load Balancing + Auto Scaling	p. 17
Module 7	VPC and Networking	p. 19
Module 8	Serverless and Messaging	p. 21
Module 9	Monitoring and Deployment	p. 23
Last page	Final cheat-sheet: capstone architecture + rapid-fire Q&A	

MODULE 1 Cloud Foundations

1. What is Cloud Computing?

Def: Renting servers, storage, databases and network **over the internet** from a provider (AWS) and paying only for what you use.

Why: No upfront hardware cost, no weeks of waiting for servers, no guessing capacity (too little = crash, too much = waste).

How: AWS owns huge data centers > **virtualization** splits 1 physical machine into many isolated VMs > you request via console / CLI / API.

Eg: Claim-status app: low traffic normally, but **after a cyclone** thousands check at once. Scale up for the spike, scale down after.

- Benefits to quote: **Elasticity**, **Pay-as-you-go**, **Global reach**, **Speed** (server in minutes), **Reliability**.

> ⚠ **INTERVIEW TRAP**
> - "Is cloud always cheaper?" NO. Steady heavy 24x7 load can be cheaper on own hardware. Cloud wins for variable load and zero upfront cost.

2. Service Models: IaaS / PaaS / SaaS

Def: How much the provider manages vs how much you manage.

- Pizza idea: on-prem = make it at home, **IaaS** = kitchen given, you cook, **PaaS** = you add toppings, **SaaS** = order finished pizza.

Model	You manage	AWS manages	AWS example
IaaS	OS, Java, app, patches	Hardware, network, virtualization	EC2
PaaS	App code + data	OS, runtime, servers	Elastic Beanstalk
SaaS	Just use it	Everything	Gmail, Salesforce

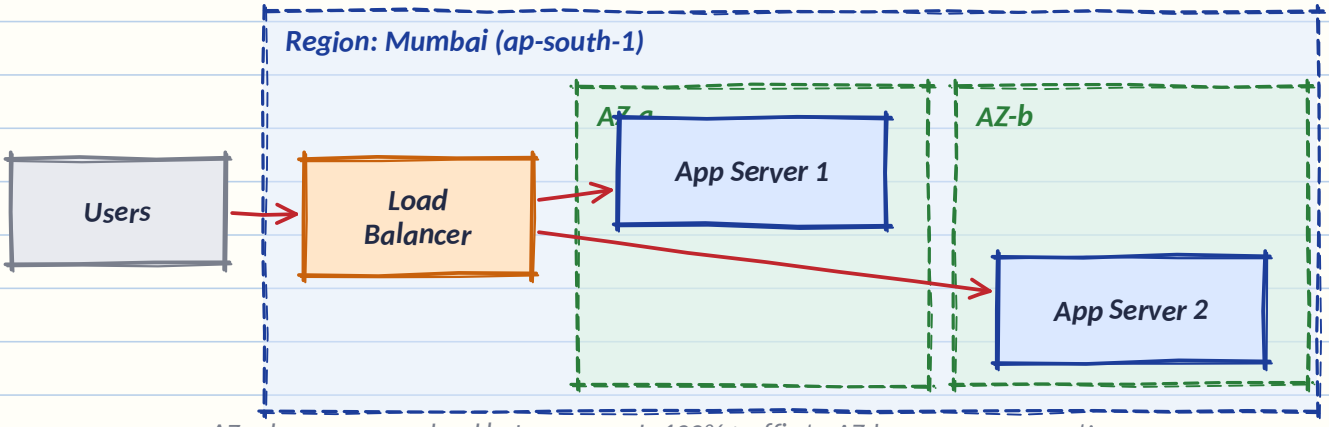
> ⚠ **INTERVIEW TRAP**
> - Don't force-label Lambda or RDS as PaaS/IaaS. They are **managed services** on a spectrum. Say that.

3. Regions and Availability Zones (AZ)

Def: **Region** = geographic area (Mumbai `ap-south-1`). **AZ** = one or more isolated data centers inside a region (Mumbai has 3).

Why: **Latency** (host near users), **fault tolerance** (one data center failing must not kill the app), **data residency** rules.

How: Run copies of the app in **2+ AZs** behind a load balancer. Both serve traffic (active-active), not idle backup.



AZ-a loses power > load balancer sends 100% traffic to AZ-b > users never notice

	Multi-AZ	Multi-Region
Protects from	One data center failing	Whole region outage / disaster
Latency between copies	Milliseconds	Higher
Cost / complexity	Moderate, standard practice	High (replication, routing)

⚠ **INTERVIEW TRAP**

- Most resources are **regional**: EC2 made in Mumbai is invisible in the Virginia console (IAM is global).
- "Multi-AZ = Multi-Region?" NO. AZ = data center failure (HA). Region = disaster recovery.

4. Shared Responsibility Model

Def: AWS secures **OF** the cloud. You secure **IN** the cloud.

AWS handles	You handle
Data centers, guards, hardware, global network, virtualization	IAM access, your data + encryption, security groups, OS patches on EC2, app code

Eg: You make an S3 bucket public by mistake and customer policy PDFs leak. **Your** fault, not AWS.

⚠ **INTERVIEW TRAP**

- "RDS is managed so everything is secure?" NO. AWS patches the engine, but access control, network exposure and data are still yours.

5. Pricing and Billing

Model	What	Use for
On-Demand	Pay per second/hour, no commitment	Learning, unpredictable load
Savings Plan / Reserved	1 or 3 yr commitment, big discount (up to ~70%)	Steady 24x7 workloads
Spot	Spare capacity, huge discount, AWS can take it back	Batch jobs. NEVER the main DB

- Hidden costs: **data transfer out**, stopped-instance **EBS volumes**, unattached **Elastic IPs**, **NAT Gateway** and **load balancers** (hourly even idle).

⚠ **INTERVIEW TRAP**

- "How do you cut AWS cost?" Strong answer: right-size, auto-scale, Savings Plans for steady load, Spot for batch, delete unused resources, budgets + alerts. Weak answer: "smaller servers".

MODULE 2 IAM

1. What is IAM?

Def: Controls **who** can do **what** on **which** resources. Authentication (who are you) + authorization (what may you do).

Why: One mistake with full access can delete a prod database or burn money. Rule: **least privilege**.

How: 4 building blocks: Users, Groups, Policies, Roles. IAM is **global** and **free**.

> ⚠ **INTERVIEW TRAP**
> - "IAM is only for users" is wrong. It also lets **apps and AWS services** talk to each other (roles).

2. Root user vs IAM user

- **Root** = email login from signup, unlimited power, cannot be restricted. Enable **MFA**, lock it away, never create root access keys.
- **IAM user** = identity per person with limited permissions. Daily work only with these (or IAM Identity Center for many people).

> ⚠ **INTERVIEW TRAP**
> - Using root for daily tasks. Root is only for rare things (close account, change plan).

3. Groups

- Collection of users. Permissions on the group apply to all members. New joiner > add to group > done.

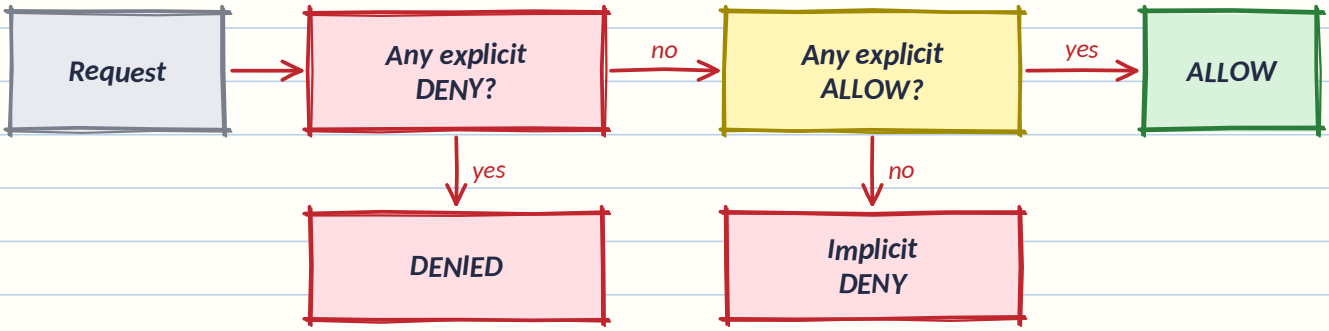
> ⚠ **INTERVIEW TRAP**
> - Groups **cannot contain other groups** and a group is **not an identity** (can't be a principal in a resource policy).

4. Policies (the rulebook)

Def: A **JSON document**: Effect (Allow/Deny), Action (`s3:GetObject`), Resource (ARN).

```
{ "Effect": "Allow",
  "Action": ["s3:GetObject", "s3:ListBucket"],
  "Resource": ["arn:aws:s3:::claims-documents",
               "arn:aws:s3:::claims-documents/*"] }
```

- Types: **AWS managed** (premade), **customer managed** (yours), **inline** (attached to one identity, avoid).
- **ARN** = unique ID of any AWS resource.



Policy evaluation: default is DENY, explicit DENY always beats ALLOW

> ⚠ **INTERVIEW TRAP**
>
> - Group allows `s3:DeleteObject` but another policy explicitly denies it? **Deny wins. Always.**
> - "Give developers AdministratorAccess" breaks least privilege.

5. Roles (most asked!)

Def: An identity **assumed temporarily** by a person or AWS service. No password, no permanent keys, hands out **temporary credentials**.

Why: Hard-coded access keys leak via Git, logs or hacked servers and never expire.

How: 1) Create role + policy 2) attach to EC2 / Lambda 3) AWS SDK auto-fetches rotating creds 4) code just calls S3.



No accessKey in application.properties. Credentials are temporary and rotate by themselves

IAM User	Role
Long-term credentials, for a person	Temporary credentials, assumed by anyone/anything trusted
Keys can leak and live forever	Keys expire automatically

⚠ **INTERVIEW TRAP**

- "How does your EC2 app access S3 securely?" > **IAM role on the instance**, never hard-coded keys.
- A role has **2 parts**: **trust policy** (who may assume it) + **permissions policy** (what it can do).

MODULE 3 EC2

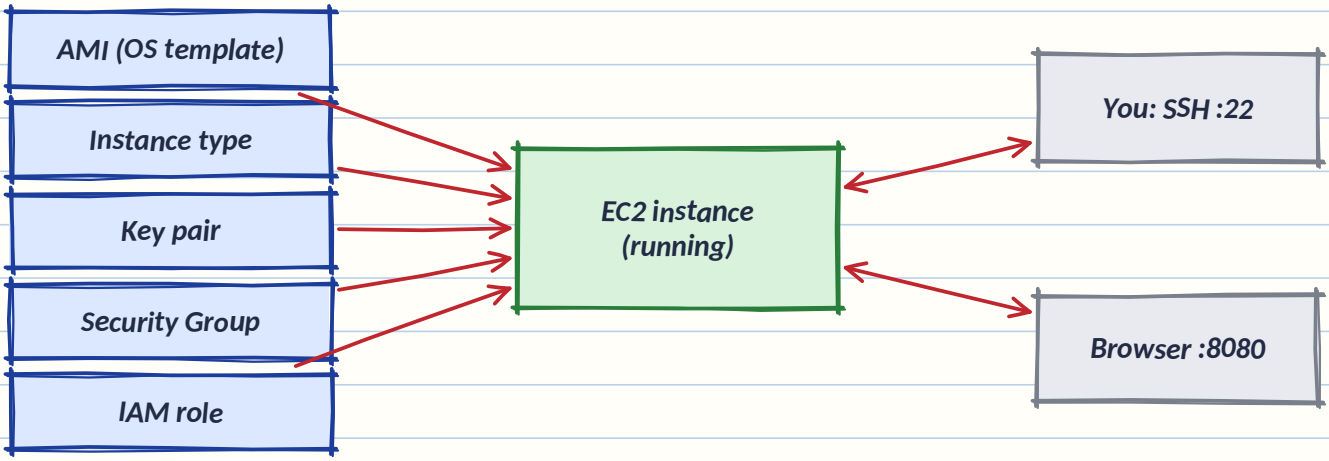
1. What is EC2?

Def: **Virtual servers** (instances) in the cloud. You pick CPU, RAM, storage, OS. Ready in about a minute.

Why: Spring Boot needs a machine. Rent per second instead of buying hardware.

How: Hypervisor slices a physical host into isolated VMs. You own OS, Java, patches, app (IaaS).

Eg: Launch Linux > SSH in > install Java 17 > copy JAR > `java -jar app.jar`.



Everything chosen at launch

> ⚠ **INTERVIEW TRAP**
> - EC2 is a **virtual machine**, not "just a server". YOU patch the OS and manage Java and the app.

2. AMI

- **Template** with OS (+ optional software). Make your own AMI from a configured server > launch 10 identical copies (Auto Scaling uses this).

> ⚠ **INTERVIEW TRAP**
> - AMIs are **region-specific**. Copy it to use in another region.

3. Instance types

- Read `t3.micro` = family **t**, generation **3**, size **micro**.

Family	Optimized for	Use
T	Burstable, cheap	Learning, small apps
M	Balanced CPU + RAM	Most backend apps
C	CPU	Heavy processing
R	RAM	Caches, in-memory DB

> ⚠ **INTERVIEW TRAP**
> - T-instance slows under constant load? **CPU credits ran out.** Right-size from real metrics.

4. Key pair

- Public key stays on instance, private `.pem` with you (downloadable **once**). Login: `ssh -i key.pem ec2-user@IP`.

> ⚠ **INTERVIEW TRAP**
> - Lost `.pem` = locked out, AWS cannot recover. Never commit it to Git.

5. Security Group = instance firewall

Type	Port	Source	Why
SSH	22	My IP only	Admin login
Custom TCP	8080	0.0.0.0/0	Spring Boot (practice)
HTTPS	443	0.0.0.0/0	Public web

- **Deny by default**, add allow rules only. **Stateful**: allowed request gets its reply automatically. Outbound open by default.

> ⚠ **INTERVIEW TRAP**
> - SSH open to 0.0.0.0/0 = classic mistake.
> - App runs but browser **times out**? Check (1) security group port (2) app listening on `0.0.0.0`, not only localhost. Timeout = network blocked, "connection refused" = nothing listening.
> - Security groups have **no deny rules** (NACLs do).

6. Storage, IPs, lifecycle

- **EBS** = virtual disk, survives stop/start, lives in **one AZ**, snapshot for backup, **billed even when stopped**.
- **Instance store** = fast host disk, **lost on stop**. Temp data only.
- Public IP **changes on stop/start**. **Elastic IP** = fixed IP (charged if unattached).



Pay compute only while running

⚠ **INTERVIEW TRAP**

- **Stop** = disk kept (pay storage). **Terminate** = deleted forever (root disk deleted by default).

MODULE 4 S3

1. What is S3?

Def: **Object storage**: files (objects) in buckets, accessed over HTTP / SDK. No servers, practically unlimited space.

Why: Files on EC2 disk die with the instance and differ per server. BLOBs in MySQL bloat the DB.

Type	Service	Like	Use
Block	EBS	Hard disk	OS, databases
File	EFS	Shared network folder	Shared files across servers
Object	S3	Drive with an API	Docs, images, backups, logs

Eg: Upload `claim-1234.pdf` > save in **S3**, store only the **S3 key + metadata in MySQL**.

> ⚠ **INTERVIEW TRAP**
> - S3 is **not a file system**. You replace a whole object, can't edit part of it. Not for databases or OS disks.

2. Buckets, objects, keys

- **Bucket** name is globally unique, data lives in **one region**. **Object** up to **5 TB**. **Key** = `claims/2026/10/claim-1234.pdf`.
- Structure is **flat**. Folders are just key **prefixes**.

> ⚠ **INTERVIEW TRAP**
> - "S3 global or regional?" Bucket **names** are global, **data** is regional.
> - Large files: use **multipart upload**.

3. Durability vs availability

- **Durability** = data not lost: **11 nines** (99.999999999%). **Availability** = reachable now: about **99.99%**.
- Copies are kept across **multiple AZs** automatically.

> ⚠ **INTERVIEW TRAP**
> - Durability does NOT protect against **your own delete**. That is what **versioning** is for.

4. Storage classes

Class	For	Note
Standard	Frequent access	Default
Intelligent-Tiering	Unknown access pattern	Auto-moves objects
Standard-IA	Rare access, fast retrieval	Cheaper storage + retrieval fee
One Zone-IA	Re-creatable data	Only 1 AZ = less resilient
Glacier (Instant / Flexible)	Archive	ms to hours retrieval
Glacier Deep Archive	Compliance, 7+ yrs	Cheapest, hours to retrieve

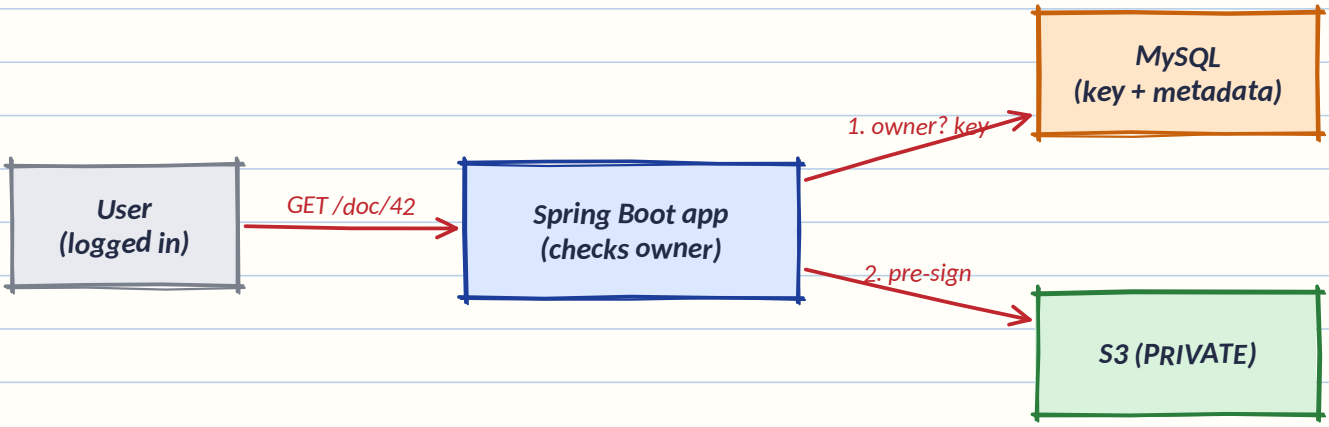
- **Lifecycle rule** example: 30 days > IA, 180 days > Glacier, 7 years > delete.

⚠ **INTERVIEW TRAP**

- IA / Glacier have **minimum storage duration + retrieval fees.** Moving hot data there costs MORE.

5. Security (most asked)

- **Private by default.** Layers: **Block Public Access** (keep ON) > **IAM policy** > **bucket policy** > ACLs (avoid, disabled by default) > **pre-signed URL**.
- Encryption: **at rest** automatic (SSE-S3), or **KMS** for audit + key control. **In transit**: HTTPS.



3. App returns 5-minute pre-signed URL > user downloads straight from S3. Bucket never public

⚠ **INTERVIEW TRAP**

- Public bucket leak = classic breach, **your** misconfiguration.
- "Private bucket but app gets Access Denied?" Check **role policy**, **bucket policy**, explicit Deny.
- Download without public bucket? **Pre-signed URL** (or CloudFront signed URL).

6. Versioning and consistency

- **Versioning** keeps every version. Delete = adds a **delete marker**. Once enabled can only be **suspended**. Pair with lifecycle (old versions cost money).
- S3 has **strong read-after-write consistency** now. "Eventual consistency" is outdated, don't say it.
- Region backup copy = **Cross-Region Replication**.

7. S3 from Spring Boot

```
S3Client s3 = S3Client.builder()
    .region(Region.AP_SOUTH_1).build();   // creds from role
s3.putObject(PutObjectRequest.builder()
    .bucket("acme-claims").key("claims/c1.pdf").build(),
    RequestBody.fromFile(path));
```

⚠ **INTERVIEW TRAP**

- "Where do creds come from?" **Default credentials provider chain** (env > config files > **instance role**). Never hard-coded.

MODULE 5 RDS and Databases

1. What is RDS?

Def: **Managed relational DB**: MySQL, PostgreSQL, MariaDB, Oracle, SQL Server, Aurora. AWS runs the machine, you run schema and queries.

Why: Self-managed MySQL on EC2 = you do backups, patching, failover, replication, disk growth.

How: AWS handles **backups, patching, Multi-AZ failover, monitoring, storage scaling, encryption**. You handle schema, indexes, query tuning, access control.

Eg: JDBC URL points to RDS endpoint instead of localhost. JPA code unchanged.

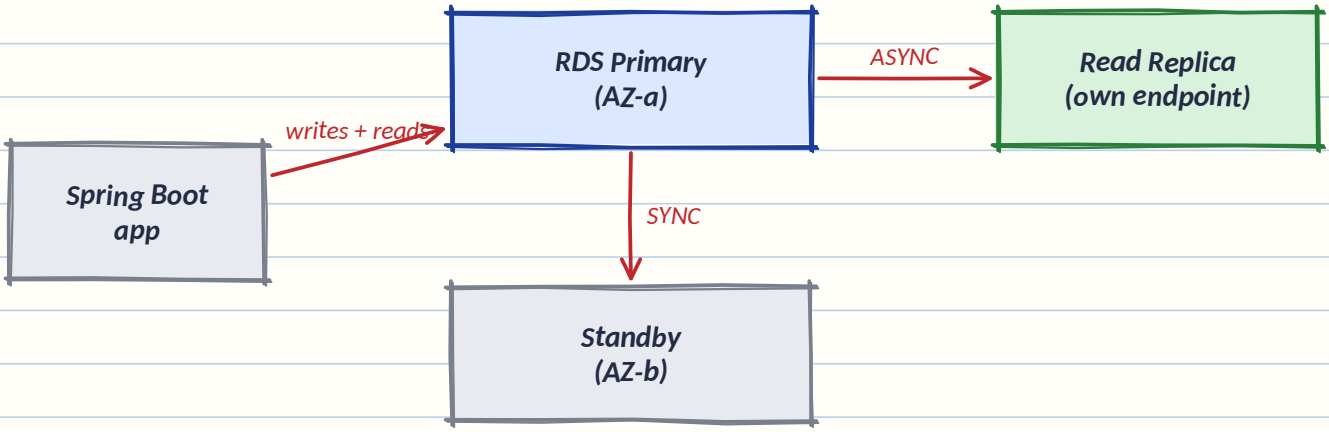
> ⚠ **INTERVIEW TRAP**
> - You **cannot SSH** into RDS. Only connect via DB port.
> - RDS is not set-and-forget. Access control and slow queries are yours.

2. Instance class and storage

- Classes like EC2: `db.t3` (dev), `db.m5` (general), `db.r5` (memory heavy). Storage: **gp3** (most), **Provisioned IOPS** (heavy), **storage autoscaling**.

> ⚠ **INTERVIEW TRAP**
> - Scaling instance class up = **brief downtime**. Disk can grow, **never shrink**.

3. Multi-AZ vs Read Replica (classic)



Writes go to primary endpoint. Reads can go to the replica endpoint. Failover repoints DNS to standby

	Multi-AZ	Read Replica
Purpose	High availability	Read scaling
Replication	**Synchronous**	**Asynchronous**
Serves traffic?	No, standby only	Yes, reads
Failover	Automatic	Manual promote

> ⚠ **INTERVIEW TRAP**
> - Replica can show **stale data (replication lag)**.
> - App must **retry/reconnect** on failover. Use the DNS endpoint, never the IP.

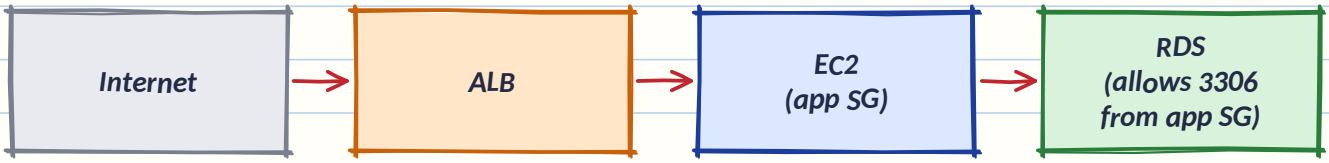
4. Backups

- **Automated**: daily snapshot + logs, retention **1 to 35 days**, **point-in-time recovery**. **Manual snapshots** kept until deleted.
- A restore **always creates a NEW instance** (new endpoint). Then switch the app.

> ⚠ **INTERVIEW TRAP**
> - "Does Multi-AZ replace backups?" **NO.** `DELETE FROM claims` hits the standby instantly. Only backups save you.
> - Delete instance = automated backups deleted too (unless final snapshot).

5. Networking and security

- Put DB in **private subnet**, **Public access = No**. DB security group allows 3306 **only from the app's security group**.
- Encrypt with **KMS**, force TLS, keep password in **Secrets Manager** (auto rotation).



> ⚠ **INTERVIEW TRAP**
> - Encryption at rest must be chosen at **creation**. Existing unencrypted DB: snapshot > encrypted copy > restore.
> - SG source = **security group**, not IP, because app IPs change when scaling.

6. Aurora vs DynamoDB

Need	Choose
Joins, transactions, SQL schema (claims, policies)	**RDS / Aurora**
Massive scale, key lookups, flexible schema, serverless	**DynamoDB**

- **Aurora**: MySQL/PostgreSQL compatible, **6 copies across 3 AZs**, up to **15 replicas**, faster failover, costs more.
- **DynamoDB**: NoSQL key-value, single-digit ms, design tables around **query patterns** (partition key).

⚠ **INTERVIEW TRAP**

- "SQL or NoSQL, which is better?" Neither. **Access pattern decides.**
- "Too many connections" after scaling out? total = pool size x instances. Use HikariCP limits or **RDS Proxy** (great with Lambda).

MODULE 6 Load Balancing + Auto Scaling

1. The big question: traffic grows 10x

Eg: One Spring Boot server at 100% CPU, site slow or down. Options:

- **Vertical scaling** = bigger instance. Simple, but limit, downtime, still single point of failure.
- **Horizontal scaling** = more instances behind a **load balancer**. Needs a **stateless** app.
- Also: **cache** (ElastiCache/Redis), **read replicas**, **CloudFront** for static, **SQS** to absorb bursts.

2. Elastic Load Balancer (ELB)

Def: Spreads incoming traffic across targets (EC2 / containers / Lambda) in **multiple AZs**.

How: Client > **listener** (port) > **rules** > **target group** > targets. **Health checks** remove unhealthy targets.

Type	Layer	Use
ALB	L7 HTTP/HTTPS	Path/host routing, microservices, Spring Boot
NLB	L4 TCP/UDP	Extreme performance, static IP
GWLB	L3	Firewall / inspection appliances
CLB	Old	Avoid

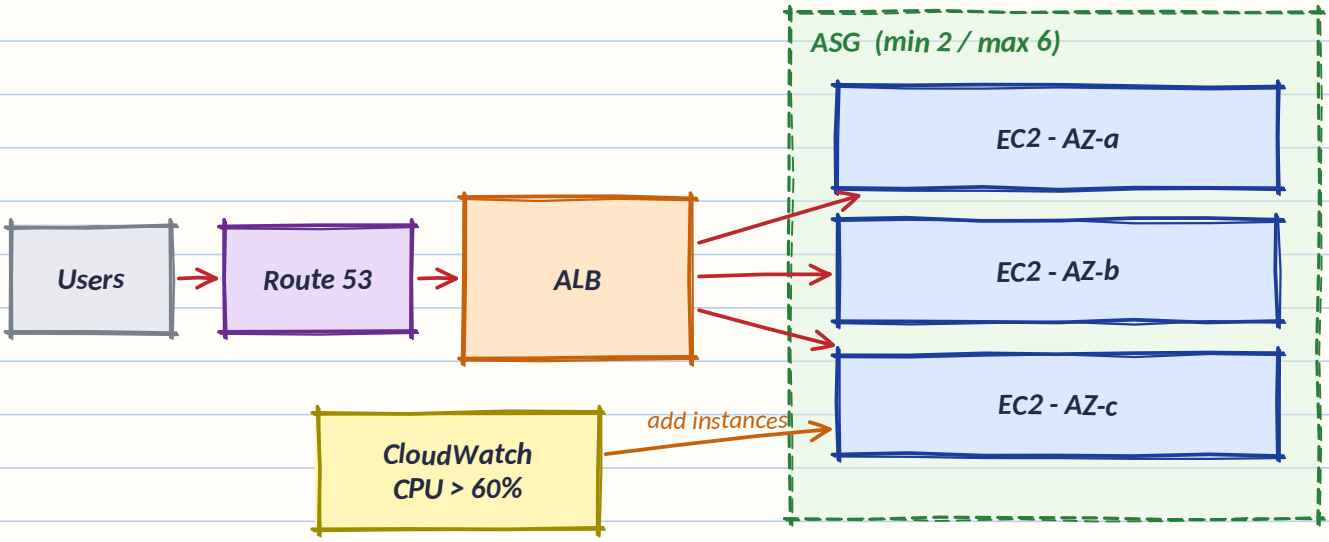
⚠ **INTERVIEW TRAP**

- Targets show **unhealthy**? Wrong health-check path/port, or instance SG doesn't allow the **ALB's SG**.
- HTTPS terminates on ALB (cert from **ACM**). Real client IP is in `X-Forwarded-For`.

3. Auto Scaling Group (ASG)

Def: A group of EC2 instances that **grows and shrinks automatically** and **replaces unhealthy ones**.

- Needs a **Launch Template** (AMI, type, SG, role, user data) + **min / desired / max** + subnets in multiple AZs.



This is the answer to "traffic 10x?"

Scaling policy	Meaning
Target tracking	Keep avg CPU at 50%. Simplest, best default
Step	Different actions at different thresholds
Scheduled	Known peak, e.g. every day 9 AM
Predictive	ML forecasts load

⚠ INTERVIEW TRAP

- **Stateless!** Session in server memory breaks on scale. Use Redis/DB or JWT. Sticky sessions = workaround only.
- Scale-in kills instances: use **connection draining** (deregistration delay) + graceful shutdown.
- `min=max=1` ASG = cheap **self-healing** single server.

MODULE 7 VPC and Networking

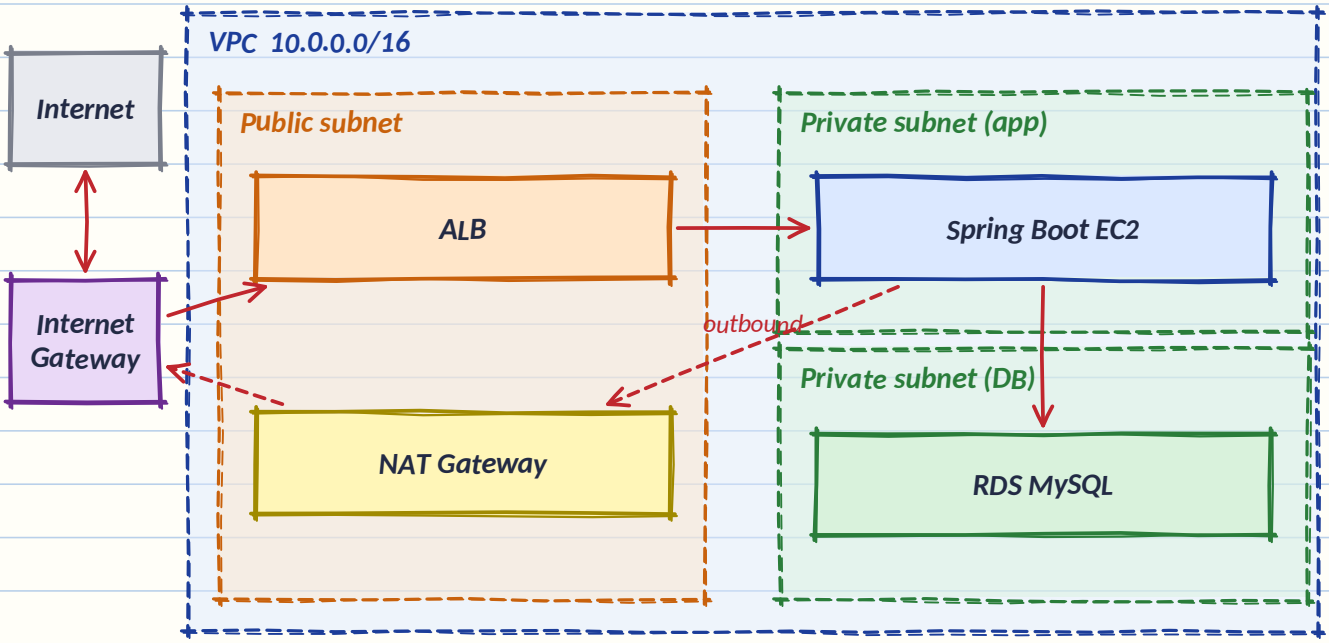
1. VPC

Def: Your own **private isolated network** in AWS. IP range by **CIDR**, e.g. `10.0.0.0/16` = 65,536 IPs.

- Lives in a **region**, spans AZs. Each region has a **default VPC** (ok for learning, not prod).

2. Subnets, IGW, NAT

- **Subnet** = slice of VPC range, lives in **ONE AZ**. AWS reserves **5 IPs** per subnet.
- **Public subnet** = route table has `0.0.0.0/0` > **Internet Gateway**. **Private** = no such route.
- **IGW**: two-way internet access for public subnets (free). **NAT Gateway**: sits in public subnet, lets private instances go **OUT** only (patches, APIs). Costs hourly + per GB.



Standard 3-tier: only ALB + NAT are public. App and DB stay private

3. Security Group vs NACL

	Security Group	NACL
Level	Instance (ENI)	Subnet
State	**Stateful**	**Stateless** (open return ports too)
Rules	Allow only	Allow **and Deny**
Evaluation	All rules together	Numbered order, first match

4. Connecting things

- **VPC endpoint**: private path to S3/DynamoDB (**gateway endpoint is free**) without internet or NAT.
- **VPC peering**: connect 2 VPCs, **not transitive**. **VPN / Direct Connect**: on-prem to AWS.
- Admin access to private servers: **bastion host** or **SSM Session Manager** (no port 22 needed).

5. Route 53

- AWS **DNS**. Records: A, CNAME, **Alias** (AWS resources, free). Routing: simple, **weighted** (canary), **latency**, **failover**, geolocation. Health checks.

⚠ **INTERVIEW TRAP**

- DB in a public subnet? Never. Private instance can't reach the internet? **NAT missing** or route missing.
- Private EC2 must read S3 without NAT cost > **gateway endpoint**.
- **CNAME not allowed at root domain** (zone apex). Use **Alias**.

MODULE 8 Serverless and Messaging

1. Lambda

Def: Run code **without managing servers**, triggered by events. Pay per request + duration (ms).

How: Upload function (Java/Python/Node) > set **memory** (CPU scales with it) + **timeout** (**max 15 min**) + **execution role** > add trigger (API Gateway, S3, SQS, EventBridge schedule).

Eg: Claim PDF uploaded to S3 > Lambda extracts data / makes thumbnail.

- **Stateless** (only `/tmp` temporary). **Cold start**: first call slower, Java worst (use **SnapStart**, lighter frameworks).

> ⚠ **INTERVIEW TRAP**
> - Lambda + RDS > too many connections. Use **RDS Proxy**.
> - NOT for: long-running jobs, steady heavy traffic, ultra-low-latency needs.

2. API Gateway

- Managed **front door for APIs**: auth (Cognito / IAM / Lambda authorizer), **throttling**, API keys, caching, validation. Types: REST, **HTTP** (cheaper), WebSocket.

> ⚠ **INTERVIEW TRAP**
> - REST API integration timeout is about **29 seconds** by default. Long work > go async (SQS).

3. SQS (queue)

Def: Managed **message queue**: producer > queue > consumer (pull).

Why: **Decouple** services, absorb spikes, retry failures.

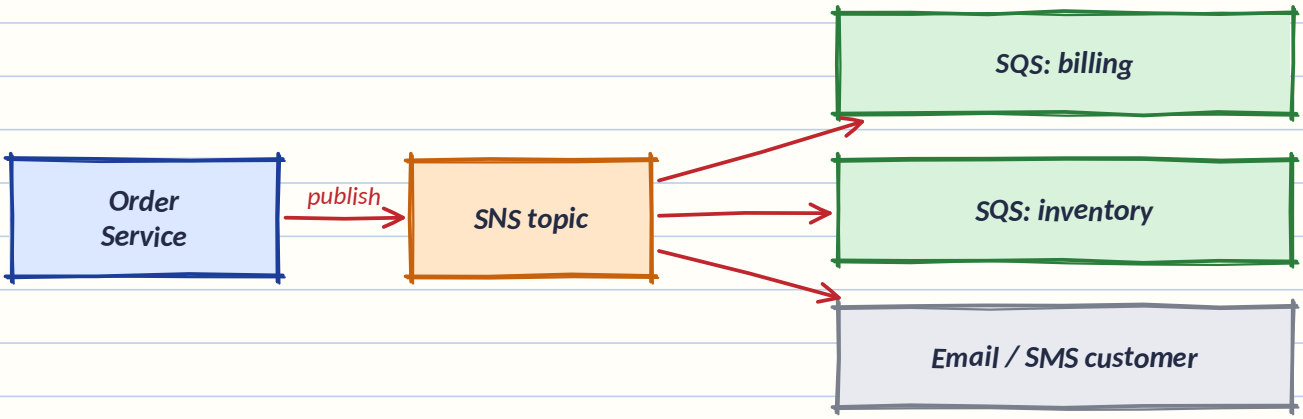
	Standard	FIFO
Delivery	At-least-once	Exactly-once processing
Order	Best effort	Strict per message group
Throughput	Nearly unlimited	Lower

- **Visibility timeout** (default 30s): message hidden while processing, **deleted on success**, reappears if not. **DLQ** collects poison messages after N retries. **Long polling** cuts empty reads.

> ⚠ **INTERVIEW TRAP**
> - Standard queue can deliver **duplicates** > make consumers **idempotent**.
> - Big payload? store in **S3**, send the key.

4. SNS (pub/sub) + EventBridge

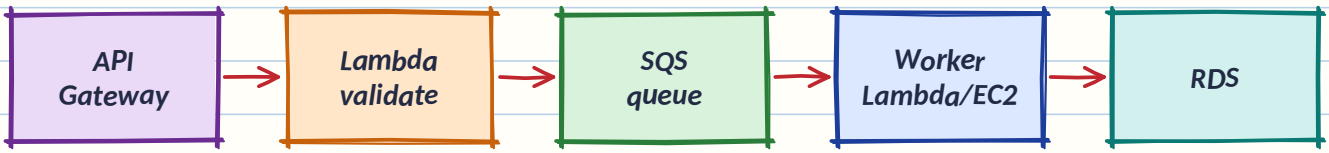
- **SNS**: one message > **many subscribers** (SQS, Lambda, email, SMS, HTTP). **Fan-out** = SNS topic > several SQS queues.
- **EventBridge**: event bus with rules, plus **scheduled cron** jobs.



Fan-out: one event, many independent consumers

	SQS	SNS
Model	Queue, **pull**	Pub/sub, **push**
Consumers	One (per message)	Many subscribers
Stores messages?	Yes, until consumed	No (retries delivery only)

5. Async claim flow (put it together)



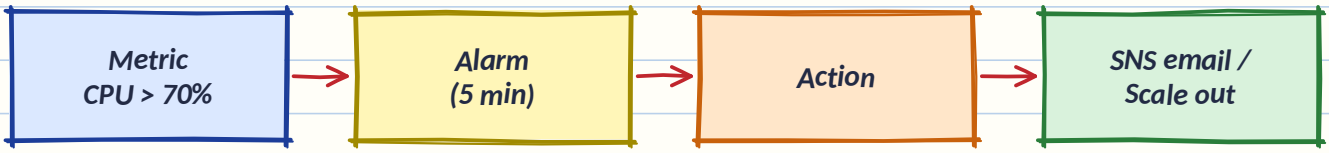
API returns 202 fast, heavy work happens behind the queue

> ⚠ **INTERVIEW TRAP**
> - Interview line: "I **decouple** with SQS so a traffic spike or a slow consumer never takes the API down."

MODULE 9 Monitoring and Deployment

1. CloudWatch

- **Metrics** (CPU, network, custom), **Logs** (log groups/streams, **Logs Insights** to query), **Alarms** (threshold > action), **Dashboards**.



- EC2 basic monitoring = **5 min**, detailed = **1 min**.

> ⚠ **INTERVIEW TRAP**
> - **Memory and disk** metrics are NOT default. Install the **CloudWatch agent**.

2. CloudWatch vs CloudTrail vs Config

Service	Answers
CloudWatch	How is it running? Metrics, logs, alarms
CloudTrail	WHO did WHAT, WHEN? Records every API call (audit)
AWS Config	How did config change? Compliance

3. Containers

```
FROM eclipse-temurin:17-jre
COPY target/app.jar app.jar
ENTRYPOINT ["java","-jar","app.jar"]
```

- Build image > push to **ECR** > run on **ECS** (**Fargate** = no servers to manage) or **EKS** (managed Kubernetes).

> ⚠ **INTERVIEW TRAP**
> - ECS vs EKS: ECS simpler and AWS-native, EKS more portable but more complex.

4. Infrastructure as Code

- **CloudFormation**: YAML/JSON templates > stacks, AWS only, auto rollback. **Terraform**: HCL, multi-cloud, **state file** (keep remote in S3 + locking).

Why: Repeatable, version-controlled, same setup for dev/test/prod, no click-ops.

> ⚠ **INTERVIEW TRAP**
> - Manual console edits cause **drift** between code and reality.

5. CI/CD and deployment strategies



Tools: GitHub Actions / Jenkins / CodePipeline + CodeBuild + CodeDeploy

Strategy	How	Good for
Rolling	Replace instances gradually	Default, cheap
Blue/Green	New environment, switch traffic	Instant rollback
Canary	Small % of users first	Risky releases

6. Secrets

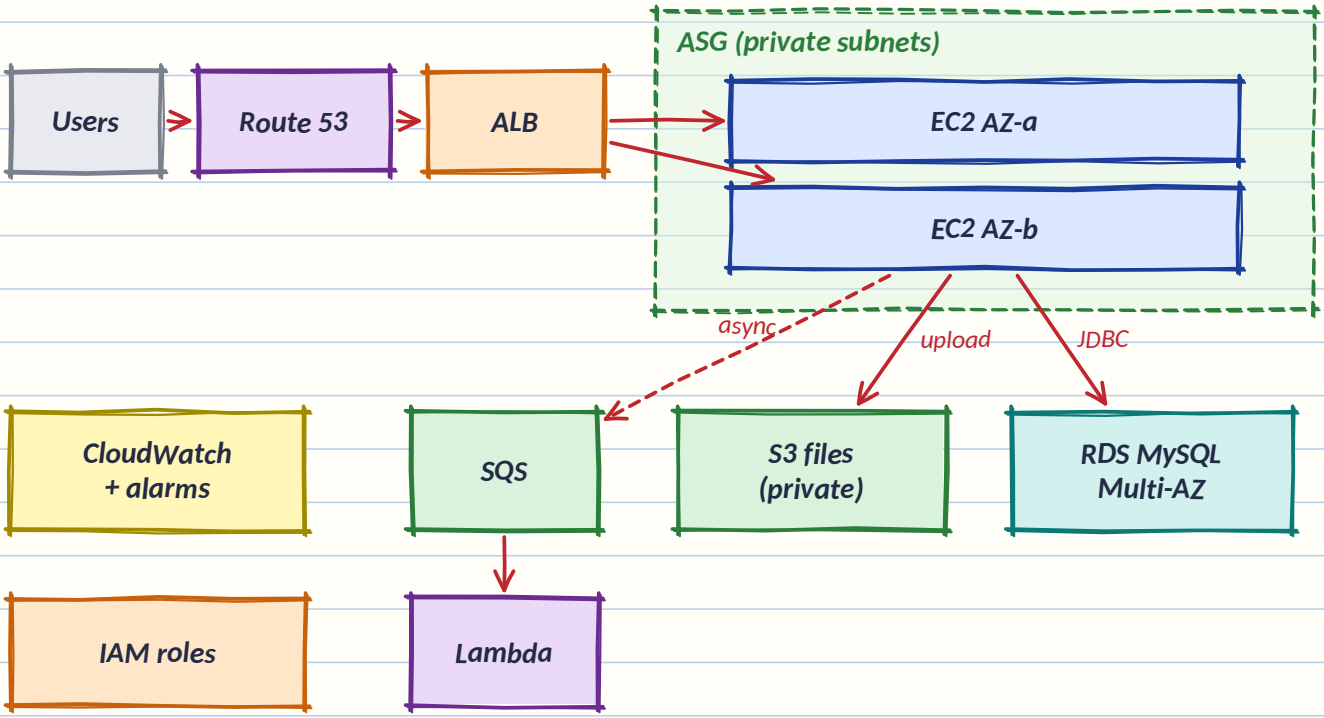
- **Secrets Manager**: auto rotation (DB passwords), costs per secret. **SSM Parameter Store**: cheap config/secrets, simple.

> ⚠ **INTERVIEW TRAP**
> - Never keep secrets in Git or `application.properties`. Pipelines use **roles / OIDC**, not long-lived keys.

FINAL SHEET

Capstone + rapid fire

The one project you can explain end to end



Spring Boot on EC2 (ASG) > RDS private > S3 via IAM role > SQS + Lambda async > CloudWatch alarms

Numbers and facts to remember

Fact	Value
S3 durability / max object	11 nines / 5 TB
Lambda max timeout	15 minutes
RDS automated backup retention	1 to 35 days
SQS visibility timeout default	30 seconds
Reserved IPs per subnet	5
/16 CIDR size	65,536 IPs
EC2 monitoring basic / detailed	5 min / 1 min
Security group default	Deny all inbound, allow all outbound

Rapid fire: say these in one breath

Question	One-line answer
Secure EC2 > S3 access?	IAM role on instance, never keys in code
Allow + Deny on same action?	Deny always wins
High availability vs DR?	Multi-AZ vs Multi-Region
Multi-AZ vs read replica?	HA, sync, standby vs read scaling, async
Does Multi-AZ replace backups?	No. Deletes replicate instantly
Download private S3 file?	Pre-signed URL
Recover deleted S3 object?	Versioning (enabled beforehand)
SG vs NACL?	Stateful instance allow-only vs stateless subnet allow+deny
Private subnet needs internet out?	NAT Gateway in public subnet
Traffic 10x?	ALB + ASG (target tracking), stateless app, cache, replicas, queue
Decouple services?	SQS (queue) / SNS (fan-out)
Who deleted that resource?	CloudTrail
Lambda + RDS problem?	Connection exhaustion > RDS Proxy
Stop vs terminate EC2?	Stop keeps disk, terminate deletes
Cut cost?	Right-size, Savings Plans, Spot, delete idle, budgets

REMEMBER

- Interview pattern: answer in 3 beats. **What it is** > **why it exists** > **a trap/trade-off you know**. The trade-off is what separates you from memorizers.